

Reinforcement Learning for Pokémon Red

Project Paper

Abstract

This project studies how reinforcement learning (RL) can be applied to the long-horizon, partially observed task of playing Pokémon Red through a Game Boy emulator. The codebase combines a PyBoy Gymnasium environment, a Stable-Baselines3 Proximal Policy Optimization (PPO) trainer, and a sweep orchestrator that manages many seeded runs while preserving resumability and post-evaluation automation. The central design idea is to replace brittle, frame-only novelty mechanisms with a structured state representation built from pixels, memory-derived game variables, and coordinate-based exploration. This paper explains the system architecture, the reward shaping logic, the training and logging pipeline, and the orchestration strategy used to run comparative sweeps over multiple configurations. It also explains the method choices in relation to standard RL practice and the broader literature on deep RL and exploration [1,2,3].

Introduction

Training an agent to play Pokémon Red is a useful test for reinforcement learning because the task combines sparse long-term objectives, partial observability, stochastic combat, and a large action space of temporally extended button presses. The agent must learn not only how to move and battle, but also how to navigate a world map, recover from setbacks, and make progress toward badges and story events. These challenges resemble the difficulties discussed in classic deep RL work such as temporal credit assignment, sparse rewards, and the need for stable policy optimization over high-dimensional observations [1,2].

The project is best understood as a systems problem layered on top of a learning problem. A policy gradient algorithm alone is not enough if the environment interface is weak, if the reward signal is too sparse to guide exploration, or if experiments cannot be reproduced after an interruption. For that reason, the repository spends substantial effort on emulator integration, logging, checkpointing, sweep management, and post-run evaluation. In other words, the project is not only about training one model, but about making a family of experiments functional and cohesive.

The repository implements two generations of the project. The older `baselines/` code path uses a frame-novelty style approach with monolithic image observations. The newer `v2/` path is the primary system and is the focus of this paper. V2 uses coordinate-based exploration, an observation space that combines pixels with structured RAM-derived features, and a cleaner reward design centered on milestone progress. In practice, this design is intended to make learning more interpretable and more compatible with batch experiment management.

The project is not only about a single training run. It also includes reproducible experiment automation: sweeps over many seeds, ranking and summarization scripts, checkpoint selection, and robustness evaluations. The current orchestrator design explicitly treats a sweep as a sequence of named runs with metadata, logs, and exit-code markers so that interrupted experiments can be resumed safely. This choice reflects a pragmatic assumption: in a long-running emulator-based RL setting, infrastructure failures are normal, so reproducibility depends as much on orchestration as on the learning algorithm.

System Overview

At a high level, each training run consists of three layers. First, `v2/baseline_fast_v2.py` configures the learning experiment and launches Stable-Baselines3 PPO. Second, `v2/red_gym_env_v2.py` exposes the emulator as a Gymnasium environment with structured observations and shaped rewards. Third, callbacks such as `v2/tensorboard_callback.py` and checkpoint saving logic record metrics and model snapshots. An orchestrator then creates many such runs and summarizes them afterward.

The data flow is intentionally explicit. The emulator emits game state in two forms: what is visible on screen (pixels) and what is stored in memory (RAM). The environment converts those sources into a learning-ready observation, the trainer maps observations to actions through PPO, and the callback stack records both training dynamics and game progress. This separation makes the code easier to inspect because each stage has one responsibility. It also makes it easier to diagnose failure modes. If learning looks poor, you can inspect the reward function. If the policy never improves, you can inspect the observations. If training crashes, you can inspect the run metadata and logs.

The emulator interface is built on the PyBoy emulator, which loads a Pokémon Red ROM and a saved state file to initialize the agent from a consistent starting point. The environment reads memory addresses for position, party state, badges, and event flags, then converts these values into an observation dictionary. The observation includes screen pixels, health, level-related features, badge bits, event bits, a local crop of the stitched global map, and a short history of recent actions. This design is deliberately hybrid. Pixels provide visual information about the world, while RAM-derived signals expose latent game state that would otherwise be difficult to infer from the frames alone. In contrast to a fully image-based setup, the observation dictionary allows the policy network to process multiple modalities with separate features. That choice is especially valuable in a partially observed game where a large fraction of the useful state is encoded in internal game memory rather than on-screen graphics.

This project also distinguishes between model quality and experiment quality. A strong checkpoint is not very useful if it cannot be recovered later, compared across seeds, or re-evaluated under perturbations. The repository therefore treats metadata as a primary artifact. Each run stores its command line, its exit code, its TensorBoard logs, and its final checkpoint path. That makes the experimental record auditable rather than implicit.

Environment Design and Reward Shaping

The core environment in `v2/red_gym_env_v2.py` is responsible for translating button presses into learning signals. The action space is discrete and corresponds to Game Boy inputs such as directional movement, A, B, and START. Each action is executed over multiple emulator ticks so that the agent experiences a temporally extended control effect rather than a single instantaneous button tap. This matches the physical semantics of controller input and makes the action space more realistic for Pokémon navigation and battle behavior.

Mathematical Notation - Observation Space

The observation is a multi-modal Gymnasium dictionary:

$$O = \{\textit{pixels}, \textit{health}, \textit{level}, \textit{badges}, \textit{events}, \textit{map}, \textit{recent_actions}\}$$

Formally, the observation vector at timestep t is expressed as:

$$O_t = (\pi_t \in \mathbb{R}^{H \times W \times C}, h_t \in \mathbb{R}, l_t \in \mathbb{N}, b_t \in \{0,1\}^8, e_t \in \{0,1\}^{|E|}, m_t \in \mathbb{R}^{H' \times W'}, a_t \in A^k)$$

where:

- π_t is the visual pixel tensor.
- h_t and l_t are the RAM-derived health and level scalars.
- b_t and e_t are the badge and event binary vectors.
- m_t is the map crop from the stitched global canvas.
- a_t is a k -length history of actions tracking short-term context from the discrete action space A .

This structural division allows a multi-input policy to learn individual sub-representations for each unique modality before combining them within a shared latent network space [1,3].

Mathematical Formulation - Structured Reward Function

The reward function is one of the most critical elements of the architecture. Instead of awarding only terminal success, the environment computes a progress vector and returns the change in its total value at each step. The reward at each step is formalised as a weighted sum of directional progress components:

$$R(s_t, s_{t+1}) = \sum_i w_i \cdot \Delta P_i(s_t, s_{t+1})$$

$$\text{for } i \in \{\text{event, badge, exploration, health, damage, level, ...}\}$$

where w_i represents the assigned component weights, and ΔP_i is the operational step change in component i between successive states. To strictly prevent progress loss (e.g., losing rewards when an agent backtracks geographically or sustains damage), each structural component explicitly tracks historical maxima:

$$P_i(s) = \max(P_i(s_{\text{prev}}), \text{current_measure}_i(s))$$

The reasoning behind this shaping is practical. In a game like Pokémon, a purely sparse objective such as “beat the game” is too delayed to support efficient policy improvement. The agent may spend millions of steps wandering without receiving a meaningful update signal. Reward shaping addresses that issue by revealing smaller achievements that correspond to viable progress. Badges, events, exploration coverage, and battle outcomes all provide partial evidence that the policy is moving toward long-horizon competence. At the same time, the design keeps the signal grounded in the actual game world rather than in an arbitrary proxy task [2].

The design has several practical consequences. First, exploration is measured by unique coordinates rather than by frame novelty. This makes the signal cheaper to compute and more semantically meaningful because Pokémon is fundamentally a spatial navigation task. Coordinate-based exploration models prevent the policy from being fooled by superfluous animation, combat visual effects, or UI shifts that are visually distinct but strategically irrelevant. Second, the `game_progress` and `event` components track maxima, which prevents the agent from losing progress after moving away from a location or state. Third, the combination of healing, damage, and battle-level shaping helps the agent learn local tactics without waiting for a badge or story event to arrive.

The stitched global map is another important design choice. Instead of treating the world as an undifferentiated pixel field, the environment maps local coordinates into a global canvas derived from known map segments. This

supports an exploration percentage metric and gives the agent a persistent notion of where it has been. The map representation is filtered by a valid-tile mask so that coverage statistics ignore padding and empty canvas regions.

Learning Algorithm and Training Loop

The main training script uses Proximal Policy Optimization, a widely used on-policy algorithm introduced by Schulman et al. [1]. PPO is a sensible choice for this domain because it is comparatively stable, requires less tuning than many off-policy alternatives, and works well with vectorized environments. The project relies on a robust general-purpose policy gradient method combined with domain-specific state and reward design.

Mathematical Formulation - Proximal Policy Optimization Loss

PPO optimizes the policy parameters by maximizing the clipped surrogate objective function:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

where:

- $r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$ is the probability ratio between the updated and old policy distributions.
- $\hat{A}_t$ is the estimated advantage at timestep t .
- ϵ is the clipping parameter (typically set to 0.2).
- $\mathbb{E}_t$ denotes the expectation over timesteps within the current policy rollout.

Mathematical Formulation - Discount Factor and Value Function

The long-horizon expected discounted return G_t is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1}$$

where $\gamma \in [0,1]$ is the temporal discount factor and r_{t+k+1} is the operational reward received at future step $t+k+1$.

The value function estimates this expected cumulative reward:

$$V(s) = \mathbb{E}[G_t | s_t = s]$$

The advantage function measures relative localized improvement:

$$A(s,a) = Q(s,a) - V(s)$$

indicating how much better taking a specific action a is compared to the baseline expected action selection under the current policy distribution.

The trainer runs multiple emulator workers using `SubprocVecEnv`, which increases sample throughput by collecting experience in parallel across CPU processes. Each worker creates its own emulator instance and contributes trajectories to the shared PPO update. This is especially important because emulator stepping is relatively expensive, and a single environment would underutilize the host machine. The resulting architecture follows the standard deep RL pattern of parallel rollouts feeding a centralized policy update [2,3].

The training pipeline also uses callbacks. A checkpoint callback saves model snapshots at fixed intervals, and a tensorboard callback records scalar summaries, histograms, and exploration imagery. This separation of concerns is deliberate. The training loop handles optimization, while the callbacks handle observability and artifact production. That makes later sweep analysis possible without changing the learning code. It also provides a natural basis for ranking runs by their final badge count, events completed, exploration coverage, or highest level reached.

The training hyperparameters reflect a balance between sample efficiency and runtime stability. The code uses vectorized environments, a moderate batch size, a small number of epochs, and a discount factor that values long-horizon progress without making immediate rewards irrelevant. These choices are large enough to learn from many parallel trajectories, but conservative enough to avoid unstable updates, which should be consistent with PPO practice in complex control tasks. The exact values are not the central contribution of the project, but their selection shows an engineering preference for controlled, repeatable training over aggressive tuning.

Another practical choice is the use of saved emulator states for initialization. Starting from a consistent state shortens the time to useful behavior and removes the need to relearn the early game introduction on every run. That matters for sweep experiments because it reduces variance and makes differences between seeds more informative. The goal is not to train an agent to start the game; the goal is to study policy learning after the environment has reached a meaningful starting point.

Logging, Summaries, and Evaluation

The project’s logging strategy is designed around reproducibility and post hoc analysis. Each run writes TensorBoard event files, stdout logs, checkpoints, and a metadata record of the exact command used. The summary script then reads final scalars from the event files and writes a `summary.csv`. This allows the project to compare seeds and configurations consistently after a sweep finishes.

The evaluation pipeline extends beyond training metrics. Two robustness evaluations are used after the best checkpoint is selected, namely input-jitter evaluation and perception-noise evaluation. Input jitter interferes with the control stream, while perception noise interferes with the observation stream. These tests are useful because they check whether the policy is genuinely robust or merely overfit to the precise training interface. In long-horizon RL tasks, robustness checks are especially valuable because brittle policies may score well in training but fail when small variances are introduced.

These evaluations also reflect a broader methodological choice: the paper is not only concerned with average return during training, but with behavior under controlled distribution shift. If a policy only succeeds when the emulator, timing, and observation stack are perfectly clean, then its practical value is limited. Jitter and noise tests probe a policy’s tolerance to differences that may arise from emulator variation, rendering differences, or real-world latency. Even though this project is still entirely within simulation, the robustness tests help separate memorization from usable skill.

The summarization workflow is similarly important. A sweep can contain many runs, each with multiple TensorBoard metrics, checkpoints, and logs. Rather than manually inspecting every run, the scripts aggregate metrics into CSV form. This allows the project to compare configurations systematically, identify the best checkpoint from a group of runs, and then apply robustness tests only to the selected model. That pipeline turns a large, noisy experimental space into a manageable set of decisions.

Orchestration and Experimental Design

The orchestration logic is implemented in `scripts/run_compare_four_configs_round_robin_v2.py`, which coordinates sweeps over multiple configurations and random seeds. The orchestrator runs one job at a time in a round-robin order across configurations, writes per-run metadata, captures logs, and stops on failure so that the problem can be inspected before proceeding. It also records exit codes in `exit_code.txt`, which makes the entire sweep resumable. If a run has already completed successfully, the orchestrator skips it when resumed.

This design solves a concrete experimental problem. When comparing multiple configurations, fairness across seeds, deterministic bookkeeping, and a way to pause or recover after a crash are fundamental. A round-robin scheduler reduces the chance that one configuration monopolizes resources or benefits from a favorable time window. The metadata files ensure that each run can be traced back to its exact command-line arguments, which is essential when a sweep has multiple modes or reward settings. The conservative stop-on-failure policy is also reasonable: a native crash can indicate a GPU, emulator, or driver issue, and continuing blindly could waste many hours of compute.

All configurations were run on the same environmental hardware setups. The only application running was the orchestrator and training pipeline. This was to avoid unfair comparisons in compute availability vs. wall clock availability. The orchestrator’s post-evaluation step is also conceptually important. It ensures that robustness tests run only after all training runs have finished successfully (i.e., first establish the training result, then measure how stable the best model is under perturbation).

The resume logic matters because long sweeps are occasionally interrupted. Power loss, emulator crashes, driver failures, and manual interruption all occur in real workflows. By writing run-level metadata and exit codes, the orchestrator creates a recoverable state machine. That is a simple idea, but it is critical for more accurate comparative studies. In the specific comparison used in this project, the orchestrator also serves as a guardrail against argument drift. The training command is assembled centrally so that all configurations share the same base hyperparameters, while only the discovered-event options vary across modes. This reduces the risk of accidentally comparing runs that differ in more than one experimental factor.

Why These Methods Make Sense

The project’s methodology is a compromise between general reinforcement learning theory and the realities of emulator-based game control. In theory, one could attempt to learn directly from raw pixels and sparse game completion rewards. In practice, this would be inefficient and difficult to debug. The project instead uses structured state, reward shaping, and carefully managed parallelism.

That compromise is appropriate because the scientific question is not whether an arbitrary neural network can fit one specific gameplay trace. The question is whether a policy can learn broadly useful behavior in a complex sequential environment. To support that question, the system reduces avoidable sources of difficulty without removing the game’s essential challenges. The agent still needs to navigate, battle, explore, and plan; it simply does so with cleaner inputs and a more informative training signal.

The project also highlights the distinction between learning dynamics and evaluation dynamics. A training policy may optimize one reward landscape, but the real question is whether its best checkpoints generalize when small perturbations are introduced. That is why the post-evaluation scripts are not an afterthought. They are part of

the methodology. Robustness is treated as a property that should be measured after training, not inferred from the training curve alone.

The project also illustrates an important lesson from applied deep RL. A “better” policy is often not the result of a single novel trick, but of a collection of aligned design decisions:

- Coordinate-based exploration is more meaningful than generic novelty for a map-based game.
- Dictionary observations are more suitable than a single composite image when the environment naturally exposes state.
- A resumable orchestrator is more useful than an ad hoc batch script when runs can last many hours and fail unpredictably.

These choices improve the odds that optimization effort is spent on learning rather than on compensating for avoidable infrastructure problems. The same lesson applies to reproducibility. If two methods are compared but one of them is harder to restart, summarize, or evaluate, then the comparison is biased by process friction rather than model quality. The orchestration and logging stack in this repository is therefore part of the experimental method, not merely a convenience layer.

Limitations and Future Work

Several limitations remain. First, reward shaping can bias the agent toward proxy objectives if the shaped terms are not well aligned with game completion. Second, the policy still depends on emulator state and saved-state initialization, so it is not a fully “from scratch” general game-playing system. Third, the current orchestration logic stops on failure rather than automatically recovering, which is conservative but not completely autonomous.

There are also modeling limitations. The policy uses structured state derived from the emulator, which is highly effective but also less general than learning solely from raw sensory input. The exploration signal is coordinate-based, which fits Pokémon Red well, but may not transfer directly to other tasks where the notion of “coverage” is less spatially grounded. In addition, reward shaping introduces domain knowledge that improves trainability but complicates claims about end-to-end discovery.

Future work could improve robustness in several directions. The reward function could be validated more systematically against downstream task success. The orchestrator could be extended with automatic retry logic for specific classes of failures. Additional evaluation protocols could include ablation studies on the observation components, such as removing the map crop or RAM-derived event bits to quantify their contribution.

Another useful extension would be a formal ablation table comparing the role of each reward term. For example, one could remove exploration reward, remove game-progress shaping, or remove the event bits from the observation and then compare both final performance and robustness to perturbation. That would help determine which parts of the method are essential and which are merely helpful implementation details.

These tests were heavily limited by my personal compute and machine availability. I wish I had a second machine to work exclusively on this problem as I would have substantially more data to review and draw conclusions from. For now, this functions as a proof of concept, and is a baseline from which I will continue to develop and explore after the semester.

Conclusion

This repository demonstrates a practical approach to training reinforcement learning agents for a complex emulator-based game. Its main contribution is not a new RL algorithm, but a carefully engineered system, namely a structured environment, shaped rewards, parallel rollout collection, checkpointing, summary statistics, and a resumable multi-configuration sweep orchestrator. Together these choices make the project reproducible, able to be debugged efficiently, and suitable for comparative experimentation.

The paper’s larger argument is that successful applied RL depends on the entire pipeline, not only the optimizer. The environment must reveal useful state, the reward must encode progress, the experiment manager must handle interruptions, and the evaluation stack must probe robustness rather than only nominal performance. In that sense, the project is a case study in how to make a difficult sequential decision problem manageable without stripping away its core complexity.

The broader lesson is that successful applied RL depends on matching method to domain. For Pokémon Red, that means combining pixel inputs with memory-derived state, using a reward function that exposes progress along meaningful subgoals, and wrapping the entire process in an experiment pipeline that can survive interruptions. In that sense, the project is a good example of how reinforcement learning research becomes effective software engineering.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [3] G. Brockman et al., “OpenAI Gym,” *arXiv:1606.01540*, 2016.
- [4] M. Pleines, D. Addis, D. Rubinstein, F. Zimmer, M. Preuss, and P. Whidden, “Pokémon Red via Reinforcement Learning,” *arXiv:2502.19920*, 2025.